

1 Signal-to-noise ratio

The signal-to-noise ratio (SNR or just SN) is a measure used in signal analysis. The measurement of a signal is always accompanied by some noise, whether it is intrinsic to the measuring device, from external sources that cannot be disentangled from the object of interest or from the medium between the source and the measuring device. The SNR can provide a measure of the intensity of the desired signal against the background signal. A common definition of SNR is

$$\text{SNR} = \frac{P_{\text{signal}}}{P_{\text{noise}}}, \quad (1)$$

where P_{signal} and P_{noise} are respectively the average power of signal and noise. By this definition one can conclude that SNR is a dimensionless quantity.

Although in images processing, another definition is commonly used:

$$\text{SNR} = \frac{S}{\sigma}, \quad (2)$$

where S is the average of the desired signal and σ is average of its uncertainty. In particular, for spectroscopy, the S is the spectral flux density measured for example in $\text{erg/s/cm}^2/\text{\AA}$. For a variable continuous signal $S(\lambda)$ which depends on the wavelength λ and a variance $\sigma^2(\lambda)$ as uncertainty, one can measure obtain the average in an interval $\Delta\lambda$ of both signal by (Cappellari and Copin, 2003):

$$S = \frac{1}{\Delta\lambda} \int_{\Delta\lambda} S(\lambda) d\lambda \quad \text{and} \quad \sigma^2 = \frac{1}{\Delta\lambda} \int_{\Delta\lambda} \sigma^2(\lambda) d\lambda. \quad (3)$$

However, spectra are most commonly obtained as discrete signals within n wavelength intervals $\delta\lambda_j$. Then the equation 3 becomes:

$$S = \frac{\sum_{j=1}^n S(\lambda_j) \delta\lambda_j}{\sum_{j=1}^n \delta\lambda_j} \quad \text{and} \quad \sigma^2 = \frac{\sum_{j=1}^n \sigma^2(\lambda_j) \delta\lambda_j}{\sum_{j=1}^n \delta\lambda_j} \quad (4)$$

for $\delta\lambda_j$ uniform throughout wavelength range then $\delta\lambda = \delta\lambda_j$ for all j . Thus

$$S = \frac{\delta\lambda \sum_{j=1}^n S(\lambda_j)}{n\delta\lambda} \quad \text{and} \quad \sigma^2 = \frac{\delta\lambda \sum_{j=1}^n \sigma^2(\lambda_j)}{n\delta\lambda}, \quad (5)$$

or yet:

$$S = \frac{1}{n} \sum_{j=1}^n S(\lambda_j) \quad \text{and} \quad \sigma^2 = \frac{1}{n} \sum_{j=1}^n \sigma^2(\lambda_j). \quad (6)$$

Finally the signal-to-noise of a sampled signal is given by

$$\text{SNR} = \left(\frac{1}{\sqrt{n}} \right) \frac{\sum_{j=1}^n S(\lambda_j)}{\sqrt{\sum_{j=1}^n \sigma^2(\lambda_j)}}. \quad (7)$$

Thereafter, we apply the previous formulas to obtain a signal-to-noise map of an observation of galaxy NGC 613 made with MUSE spectrograph (Figure 1).

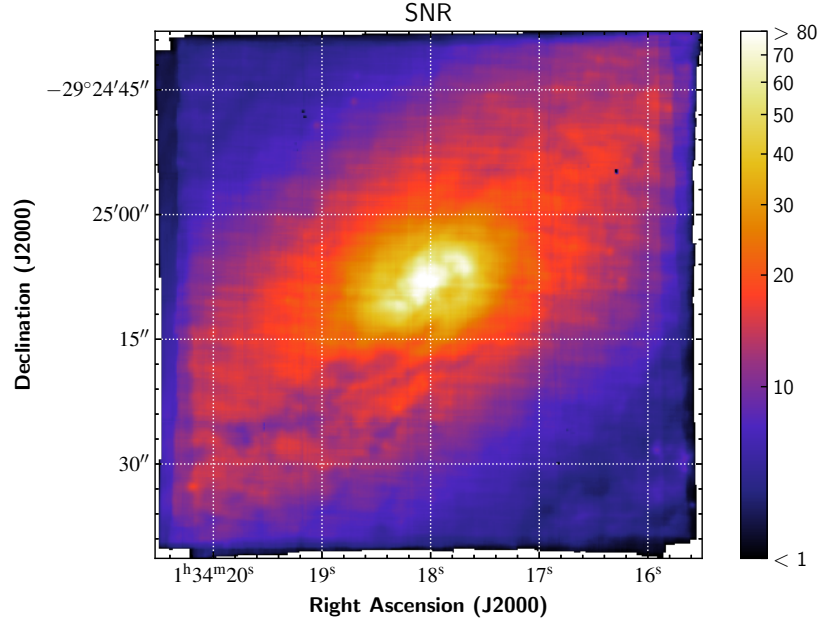


Figure 1: Signal-to-noise map of galaxy NGC 613. The observation was done with MUSE spectrograph. The image was enhanced (Gonzalez and Woods, 2018) by clipping the intensity range between 1 and 80. The image was colored was produced by mapping the intensity of each pixel $I(x, y)$ with a color scale using an arcsinh function (Lupton et al., 2004).

2 Spectrum processing

In this section, we will test the performance of different implementations for processing signals. We will analyze implementations for resampling, rebinning and convolution of signals. For this we will use one of the spectra close to the central region of NGC 613 obtained with MUSE as the signal to be processed. The programs to be used are the following:

- Our implementation;
- `SpectRes` (Carnall, 2017) which is a function¹ for resampling that conserves the flux. `SpectRes` code is used in `specutils`² which is one of Astropy components (Astropy Collaboration et al., 2013, 2018), thus being a widespread application;
- `pPXF` util (Cappellari, 2017; Cappellari and Copin, 2003) is part of the `pPXF`³ (Penalized Pixel-Fitting, a code for spectrum fitting) being composed of auxiliary functions for the preparation of spectra.

All programs used are open source python applications.

2.1 Resampling

First we're going to evaluate the performance of different applications that perform signal resampling. The MUSE spectra are sampled in a linear scale with uniform steps of $1.25 \text{ \AA}$. The MUSE spectrum used is shown in blue in Figure 2.1 in units of $10^{-20} \text{ erg/s/cm}^2 \text{ \AA}$, being its flux equal to $4.88 \times 10^{-14} \text{ erg/s/cm}^2$.

The implementation of `pPXF` util allows only the conversion from linear scale to natural logarithmic scale. Thus for this test we use all implementations for the same conversion of linear scale to natural logarithmic scale. For `pPXF` util, the `flux` keyword is set to `True` thus enabling the option that guarantees the flux conservation. The resampling made both with our implementation and `SpectRes` and `pPXF` util was done in natural logarithmic scale with uniform steps in $\ln$ -scale of 78.89 km s^{-1} . The result of resampling for all applications is shown in Figure 2.1. It can be noted that the flux was conserved only with `SpectRes` and

¹<https://www.github.com/ACCarnall/SpectRes>

²<https://specutils.readthedocs.io/en/stable/manipulation.htm#resampling1>

³<https://www-astro.physics.ox.ac.uk/~mxc/software/#ppxf>

with our implementation. These two methods achieve such similar results that it is difficult to distinguish the two signal. Therefore we employed a wider line for our implementation.

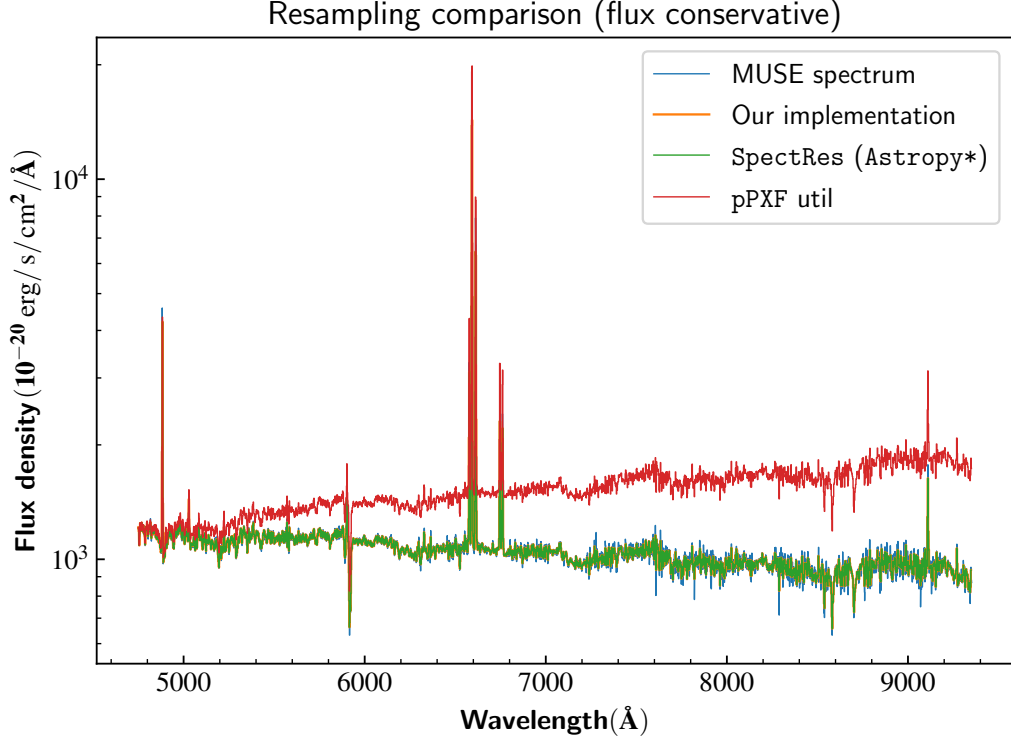


Figure 2: Resampling result for the three methods: Our implementation(orange), pPXF(red) and SpectRes - attached to Astropy(green), compared to MUSE spectrum (blue).

We measure the execution time of these programs. The results are summarized in Table 1. In the last column, we show the average execution time after 1000 execution rounds. The SpectRes implementation despite conserving the flux is two orders of magnitude slower than our implementation and pPXF util. For a single spectrum this is not an issue, but for tens of thousands of spectra, it can have a big impact on runtime.

Table 1: Results of comparison of three different implementations of resampling. For pPXF util, the `flux` keyword is set to `True`. In the central column the flux integrated over the wavelength range. In the last column, we show the average execution time after 1000 rounds of execution.

	Flux (10^{-14} erg/s/cm ²)	Average runtime (ms)
MUSE	4.88	-
Our implementation	4.88	0.226
SpectRes	4.88	70.759
pPXF util	7.14	0.296

2.2 Rebinning

2.3 Convolution with MUSE LSF

References

Astropy Collaboration, Price-Whelan, A. M., Sipőcz, B. M., Günther, H. M., Lim, P. L., Crawford, S. M., Conseil, S., Shupe, D. L., Craig, M. W., Dencheva, N., Ginsburg, A., VanderPlas, J. T., Bradley, L. D., Pérez-Suárez, D., de Val-Borro, M., Aldcroft, T. L., Cruz, K. L., Robitaille, T. P., Tollerud, E. J., Ardelean, C., Babej, T., Bach, Y. P., Bachetti, M., Bakanov, A. V., Bamford, S. P., Barentsen, G., Barmby, P., Baumbach, A., Berry, K. L., Biscani, F., Boquien, M., Bostroem, K. A., Bouma, L. G., Brammer, G. B., Bray, E. M., Breytenbach, H., Buddelmeijer, H., Burke, D. J., Calderone, G., Cano Rodríguez, J. L., Cara, M., Cardoso, J. V. M., Cheedella, S., Copin, Y., Corrales, L., Crichton, D., D’Avella, D., Deil, C., Depagne, É., Dietrich, J. P., Donath, A., Droettboom, M., Earl, N., Erben, T., Fabbro, S., Ferreira, L. A., Finethy, T., Fox, R. T., Garrison, L. H., Gibbons, S. L. J., Goldstein, D. A., Gommers, R., Greco, J. P., Greenfield, P., Groener, A. M., Grollier, F., Hagen, A., Hirst, P., Homeier, D., Horton, A. J., Hosseinzadeh, G., Hu, L., Hunkeler, J. S., Ivezić, Ž., Jain, A., Jenness, T., Kanarek, G., Kendrew, S., Kern, N. S., Kerzendorf, W. E., Khvalko, A., King, J., Kirkby, D., Kulkarni, A. M., Kumar, A., Lee, A., Lenz, D., Littlefair, S. P., Ma, Z., Macleod, D. M., Mastropietro, M., McCully, C., Montagnac, S., Morris, B. M., Mueller, M., Mumford, S. J., Muna, D., Murphy, N. A., Nelson, S., Nguyen, G. H., Ninan, J. P., Nöthe, M., Ogaz, S., Oh, S., Parejko, J. K., Parley, N., Pascual, S., Patil, R., Patil, A. A., Plunkett, A. L.,

- Prochaska, J. X., Rastogi, T., Reddy Janga, V., Sabater, J., Sakurikar, P., Seifert, M., Sherbert, L. E., Sherwood-Taylor, H., Shih, A. Y., Sick, J., Silbiger, M. T., Singanamalla, S., Singer, L. P., Sladen, P. H., Sooley, K. A., Sornarajah, S., Streicher, O., Teuben, P., Thomas, S. W., Tremblay, G. R., Turner, J. E. H., Terrón, V., van Kerkwijk, M. H., de la Vega, A., Watkins, L. L., Weaver, B. A., Whitmore, J. B., Woillez, J., Zabalza, V., and Astropy Contributors (2018). The Astropy Project: Building an Open-science Project and Status of the v2.0 Core Package. *AJ*, 156(3):123.
- Astropy Collaboration, Robitaille, T. P., Tollerud, E. J., Greenfield, P., Droettboom, M., Bray, E., Aldcroft, T., Davis, M., Ginsburg, A., Price-Whelan, A. M., Kerzendorf, W. E., Conley, A., Crighton, N., Barbary, K., Muna, D., Ferguson, H., Grollier, F., Parikh, M. M., Nair, P. H., Unther, H. M., Deil, C., Woillez, J., Conseil, S., Kramer, R., Turner, J. E. H., Singer, L., Fox, R., Weaver, B. A., Zabalza, V., Edwards, Z. I., Azalee Bostroem, K., Burke, D. J., Casey, A. R., Crawford, S. M., Dencheva, N., Ely, J., Jenness, T., Labrie, K., Lim, P. L., Pierfederici, F., Pontzen, A., Ptak, A., Refsdal, B., Servillat, M., and Streicher, O. (2013). Astropy: A community Python package for astronomy. *A&A*, 558:A33.
- Cappellari, M. (2017). Improving the full spectrum fitting method: accurate convolution with Gauss-Hermite functions. *MNRAS*, 466(1):798–811.
- Cappellari, M. and Copin, Y. (2003). Adaptive spatial binning of integral-field spectroscopic data using Voronoi tessellations. *MNRAS*, 342(2):345–354.
- Carnall, A. C. (2017). Spectres: A fast spectral resampling tool in python.
- Gonzalez, R. and Woods, R. (2018). *Digital Image Processing*. Pearson.
- Lupton, R., Blanton, M. R., Fekete, G., Hogg, D. W., O’Mullane, W., Szalay, A., and Wherry, N. (2004). Preparing Red-Green-Blue Images from CCD Data. *PASP*, 116(816):133–137.